Artificial intelligence / Machine learning
Welcome
- Introduction
- Data science for machine learning
- Feature engineering
- Supervised learning
- Unsupervised learning
- Semi-supervised learning
- Reinforcement learning
- Deep learning
- Generative AI

What is fine-tuning?

Published 15 March 2024

eBook
Data science and MLOps for data leaders
Join forces with other leaders to drive the three essential pillars of MLOps and trustworthy AI: trust in data, trust in models and trust in processes.
Read the ebook →

By Dave Bergmann

What is fine-tuning?

Fine-tuning in machine learning is the process of adapting a pre-trained model for specific tasks or use cases. It has become a fundamental deep learning technique, particularly in the training process of foundation models used for generative AI.

Fine-tuning could be considered a subset of the broader technique of *transfer learning*: the practice of leveraging knowledge an existing model has already learned as the starting point for learning new tasks.

The intuition behind fine-tuning is that, essentially, it's easier and cheaper to hone the capabilities of a pre-trained base model that has already acquired broad learnings relevant to the task at hand than it is to train a new model from scratch for that specific purpose. This is especially true for deep learning models with millions or even billions of parameters, like the large language models (LLMs) that have risen to prominence in the field of natural language processing (NLP) or the complex convolutional neural networks (CNNs) and vision transformers (ViTs) used for computer vision tasks like image classification, object detection or image segmentation.

By leveraging prior model training through transfer learning, fine-tuning can reduce the amount of expensive computing power and labeled data needed to obtain large models tailored to niche use cases and business needs. For example, fine-tuning can be used to simply adjust the conversational tone of a pre-trained LLM or the illustration style of a pre-trained image generation model; it could also be used to supplement learnings from a model's original training dataset with proprietary data or specialized, domain-specific knowledge.

Fine-tuning thus plays an important role in the real-world application of machine learning models, helping democratize access to and customization of sophisticated models.

The latest AI trends, brought to you by experts
Get curated insights on the most important—and intriguing—AI news. Subscribe to our weekly Think newsletter. See the IBM Privacy Statement.
First name*
Last name*
Business email*
* All fields required
Your subscription will be delivered in English. You will find an unsubscribe link in every newsletter. Refer to our IBM Privacy Statement for more information.
Subscribe

Fine-tuning vs. training

While fine-tuning is ostensibly a technique used in model training, it's a process distinct from what is conventionally called "training." For the sake of disambiguation, data scientists typically refer to the latter as *pre-training* in this context.

(Pre-)Training

At the onset of training (or, in this context, *pre-training*), the model has not yet "learned" anything. Training begins with a random initialization of *model parameters*—the varying weights and biases applied to the mathematical operations occurring at each node in the neural network.

Training occurs iteratively in two phases: in a *forward pass*, the model makes predictions for a batch of sample inputs from the training dataset, and a *loss function* measures the difference (or *loss*) between the model's predictions for each input and the "correct" answers (or *ground truth*); during *backpropagation*, an optimization algorithm—typically gradient descent—is used to adjust model weights across the network to reduce loss. These adjustments to model weights are how the model "learns." The process is repeated across multiple training epochs until the model is deemed to be sufficiently trained.

Conventional *supervised learning*, which is typically used to pre-train models for computer vision tasks like image classification, object detection or image segmentation, uses labeled data: labels (or *annotations*) provide both the range of possible answers and the ground truth output for each sample.

LLMs are typically pre-trained through *self-supervised learning (SSL)*, in which models learn through *pretext tasks* that are designed to derive ground truth from the inherent structure of unlabeled data. These pretext tasks impart knowledge useful for *downstream tasks*. They typically take one of two approaches:

- *Self-prediction*: masking some part of the original input and tasking the model with reconstructing it. This is the dominant mode of training for LLMs.
- *Contrastive learning*: training models to learn similar embeddings for related inputs and different embeddings for unrelated inputs. This is used prominently in computer vision models designed for few-shot or zero-shot learning, like Contrasting Language-Image Pretraining (CLIP).

SSL thus allows for the use of massively large datasets in training without the burden of having to annotate millions or billions of data points. This saves a tremendous amount of labor, but nevertheless requires huge computational resources.

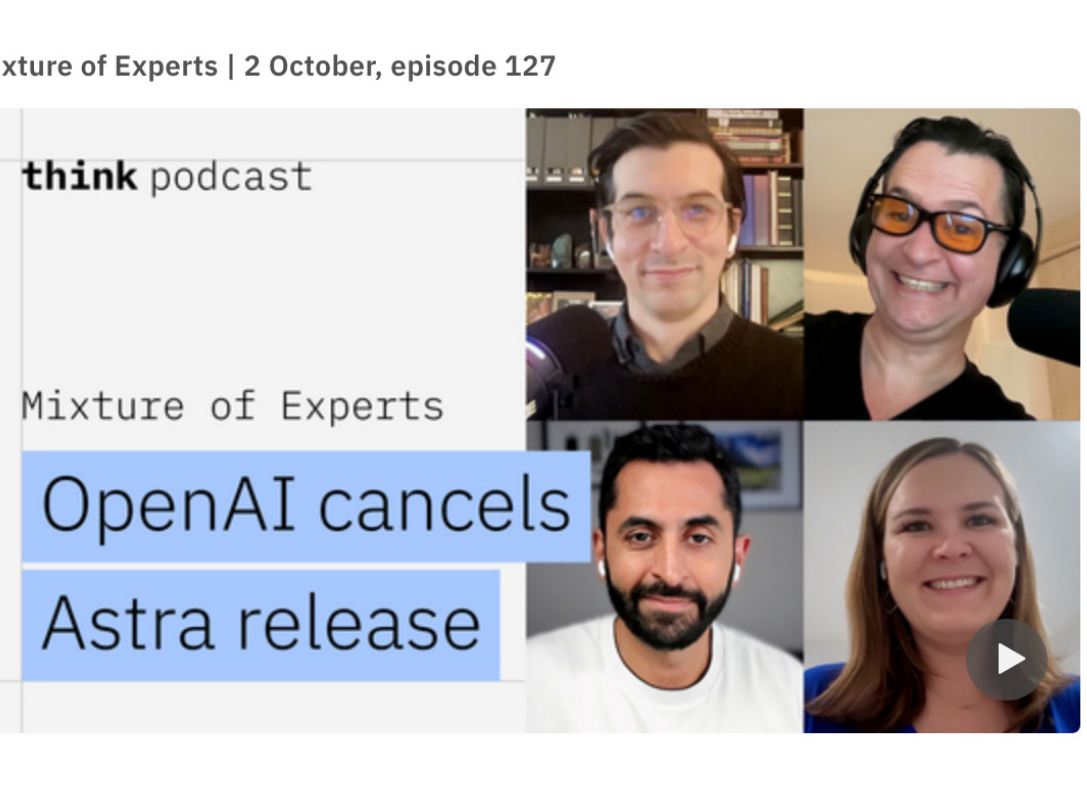
Fine-tuning

Conversely, *fine-tuning* entails techniques to further train a model whose weights have already been updated through prior training. Using the base model's previous knowledge as a starting point, fine-tuning tailors the model by training it on a smaller, task-specific dataset.

While that task-specific dataset could theoretically have been used for the initial training, training a large model from scratch on a small dataset risks *overfitting*: the model might learn to perform well on the training examples, but generalize poorly to new data. This would render the model ill-suited to its given task and defeat the purpose of model training.

Fine-tuning thus provides the best of both worlds: leveraging the broad knowledge and stability gained from pre-training on a massive set of data and honing the model's understanding of more detailed, specific concepts. Given the increasing prowess of open source foundation models, the benefits can often be enjoyed without any of the financial, computational or logistical burden of pre-training.

Mixture of Experts | 2 October, episode 127



Your weekly news podcast for AI enthusiasts
Hear from industry experts on the latest in AI news, listen to the Mixture of Experts podcast. New episodes on Fridays at 6 AM EST.
Go to episodes →

How does fine-tuning work?

Fine-tuning uses the weights of a pre-trained model as a starting point for further training on a smaller dataset of examples that more directly reflect the specific tasks and use cases the model will be utilized for. It typically entails supervised learning, but can also involve reinforcement learning, self-supervised learning or semi-supervised learning.

The datasets used for fine-tuning convey the specific domain knowledge, style, tasks or use cases for which the pre-trained model is being fine-tuned. For example:

- An LLM pre-trained for general language might be fine-tuned for coding with a new dataset containing relevant programming requests and corresponding code snippets for each.
- An image classification model used to identify certain species of birds can learn new species through additional labeled training samples.
- An LLM can learn to emulate a specific writing style through self-supervised learning on sample texts representing that style.

Semi-supervised learning, a subset of machine learning that incorporates both labeled and unlabeled data, is advantageous when the scenario calls for supervised learning but suitable labeled examples are scarce. Semi-supervised fine-tuning has yielded promising results for both computer vision[1] and NLP[2] tasks and helps reduce the burden of acquiring a sufficient amount of labeled data.

Fine-tuning can be used to update the weights of the entire network, but for practical reasons this is not always the case. There exist a wide variety of alternate fine-tuning methods, often referred to under the umbrella term of *parameter-efficient fine-tuning* (PEFT), that update only a select subset of model parameters. PEFT methods, which are explored later in this section, can decrease computational demands and reduces catastrophic forgetting—the phenomenon in which fine-tuning causes the loss or destabilization of the model's core knowledge—often without meaningful compromises in performance.

Given the wide variety of fine-tuning techniques and the many variables inherent to each, achieving ideal model performance often requires multiple iterations of training strategies and setups, adjusting datasets and hyperparameters like batch size, learning rate and regularization terms until a satisfactory outcome—per whichever metrics are most relevant to your use case—has been reached.

Full fine-tuning

The most conceptually straightforward means of fine-tuning is to simply update the entire neural network. This simple methodology essentially resembles the pre-training process: the only fundamental differences between the full fine-tuning and pre-training processes are the dataset being used and the initial state of the model's parameters.

To avoid destabilizing changes from the fine-tuning process, certain *hyperparameters*—model attributes that influence the learning process but are not themselves learnable parameters—might be adjusted relative to their specifications during pre-training: for example, a smaller *learning rate* (which reduces the magnitude of each update to model weights) is less likely to lead to catastrophic forgetting.

Parameter efficient fine-tuning (PEFT)

Full fine-tuning, like the pre-training process it resembles, is very computationally demanding. For modern deep learning models with hundreds of millions or even many billions of parameters, it's often prohibitively costly and impractical.

Parameter efficient fine-tuning (PEFT) encompasses a range of methods to reduce the number of trainable parameters that need to be updated in order to effectively adapt a large pre-trained model to specific downstream applications. In doing so, PEFT significantly decreases the computational resources and memory storage needed to yield an effectively fine-tuned model. PEFT methods have often been demonstrated to be more stable than full fine-tuning methods, particularly for NLP use cases.[3]

Partial fine-tuning

Also called *selective fine-tuning*, partial fine-tuning methods aim to reduce computational demands by updating only the select subset of pre-trained parameters most critical to model performance on relevant downstream tasks. The remaining parameters are "frozen," ensuring that they will not be changed.

The most intuitive partial fine-tuning approach is to update only the outer layers of the neural network. In most model architectures, the inner layers of the model (closest to the input layer) capture only broad, generic features: for example, in a CNN used for image classification, early layers typically discern edges and textures; each subsequent layer discerns progressively finer features until final classification is predicted at the outermost layer. Generally speaking, the more similar the new task (for which the model is being fine-tuned) is to the original task, the more useful the pre-trained weights of the inner layers will already be for this new, related task—and thus the fewer layers need to be updated).

Other partial fine-tuning methods including updating only the layer-wide bias terms of the model (rather than the node-specific weights)[4] and "sparse" fine-tuning methods that update only a select subset of overall weights throughout the model.[5]

Additive fine-tuning

Rather than fine-tuning the existing parameters of a pre-trained model, additive methods add extra parameters or layers to the model, freeze the existing pre-trained weights, and train only those new components. This approach helps retain stability of the model by ensuring that the original pre-trained weights remain unchanged.

While this can increase training time, it significantly reduces memory requirements because there are far fewer gradients and optimization states to store: according to Lialin, et al, training all of a model's parameters requires 12–20 times more GPU memory than the model weights alone.[6] Further memory savings can be achieved through *quantization* of the frozen model weights: a reduction in the precision used to represent model parameters, conceptually similar to lowering the bitrate of an audio file.

One sub-branch of additive methods is *prompt tuning*. Conceptually, it's similar to prompt engineering, which refers to tailoring "hard prompts"—that is, prompts written by a human in natural language—to guide the model toward the desired output, such as by specifying a certain tone or by providing examples that facilitate few-shot learning. Prompt tuning introduces AI-authored *soft prompts*: learnable vector embeddings that are concatenated to the user's hard prompt. Rather than retraining the model, prompt tuning entails freezing model weights and instead trains the soft prompt itself. Fast and efficient, prompt tuning allows for models to more easily switch between specific tasks, albeit with a tradeoff in interpretability.

Adapters

Another subset of additive fine-tuning injects *adapter modules*—new, task-specific layers added to the neural network—and trains these adapter modules in lieu of fine-tuning any of the pre-trained model weights (which are frozen). According to the original paper, which measured results on the BERT masked language model, adapters attained performance equivalent to that of full fine-tuning while training only 3.6% as many parameters.[7]

Reparameterization

Reparameterization-based methods like *Low Rank Adaptation (LoRA)* leverage low-rank transformation of high-dimensional matrices (like the massive matrix of pre-trained model weights in a transformer model). These low-rank representations omit inconsequential higher-dimensional information in order to capture the underlying low-dimensional structure of model weights, greatly reducing the number of trainable parameters. This dramatically speeds up fine-tuning and reduces memory needed to store model updates.

LoRA eschews direct optimization of the matrix of model weights and instead optimizes a matrix of updates to model weights (or *delta weights*), which is inserted into the model. That matrix of weight updates is, in turn, represented as two smaller (i.e., *lower rank*) matrices, greatly reducing the number of parameters to be updated—which, in turn, dramatically speeds up fine-tuning and reduces memory needed to store model updates. The pre-trained model weights themselves remain frozen.

An added benefit of LoRA is that, since what's being optimized and stored are not new model weights but rather the difference (or delta) between the original pre-trained weights and fine-tuned weights, different task-specific LoRAs can be "swapped in" as needed to adapt the pre-trained model—whose actual parameters remain unchanged—to a given use case.

A variety of LoRA derivatives has been developed, such as *QLoRA*, which further reduces computational complexity by quantizing the transformer model prior to LoRA.

Fine-tuning large language models

Fine-tuning is an essential part of the LLM development cycle, allowing the raw linguistic capabilities of base foundation models to be adapted for a variety of use cases, from chatbots to coding to other domains both creative and technical.

LLMs are pre-trained using self-supervised learning on a massive corpus of unlabeled data. Autoregressive language models, like OpenAI's GPT, Google's Gemini or Meta's Llama models, are trained to simply predict the next word(s) in a sequence until it's complete. In pre-training, models are provided the beginning of a sample sentence drawn from the training data and repeatedly tasked with predicting the next word in the sequence until the end of the sample. For each prediction, the actual next word of the original sample sentence serves as ground truth.

While this pre-training yields powerful text generation capabilities, it does not yield any actual understanding of a user's intent. On a fundamental level, autoregressive LLMs do not actually answer a prompt; they only *append text to it*. Without very specific guidance in the form of prompt engineering, a pre-trained LLM (that has not been fine-tuned) simply predicts, in a grammatically coherent way, what might be the next word(s) in a given sequence that is initiated by the prompt. If prompted with "*teach me how to make a resumé*," an LLM might respond with "*using Microsoft Word*." It's a valid way to complete the sentence, but not aligned with user's goal. The model might already have a substantial knowledge of resumé writing gleaned from relevant content included in its pre-training corpus, but without fine-tuning this knowledge might not be accessed.

The fine-tuning process thus serves a crucial role in not only tailoring foundation models for your or your business's unique tone and use cases, but in making them altogether suitable for practical usage.

Instruction tuning

Instruction tuning is a subset of supervised fine-tuning (SFT), often used to fine-tune LLMs for chatbot usage, that primes the LLM to generate responses that more directly address user needs: in other words, to better follow instructions. Labeled examples, following the format (*prompt, response*)—in which the prompt examples comprise instruction-oriented tasks, like "*translate the following sentence from English to Spanish*" or "*classify the following sentence as Positive or Negative*"—demonstrate how to respond to prompts representing a variety of use cases, like question answering, summarization or translation. In updating model weights to minimize the loss between model outputs and the labeled samples, the LLM learns to append text to prompts in a more useful way and better follow instructions in general.

Continuing the earlier prompt example of "*teach me how to write a resumé*," the dataset used for SFT could contain a number of (*prompt, response*) pairs demonstrating that the desired way to respond to prompts beginning with "*teach me how to*" is to provide step by step suggestions, rather than merely complete the sentence.

Reinforcement learning from human feedback (RLHF)

While instruction tuning can teach the model tangible, straightforward behaviors like how to structure its responses, it can be prohibitively laborious and difficult to teach abstract human qualities like helpfulness, factual accuracy, humor or empathy through labeled examples.

To better align model outputs with ideal human behavior, especially for conversational use cases like chatbots, SFT may be supplemented with reinforcement learning—more specifically, reinforcement learning from human feedback (RLHF). RLHF, also called *reinforcement learning from human preferences*, helps fine-tune models for qualities that are complex, ill-defined or difficult to specify through discrete examples.

Consider comedy: to teach a model to be "funny" with SFT not only requires the cost and labor of writing (or acquiring) enough jokes to constitute a learnable pattern, but also requires that what a given data scientist thinks is funny aligns with what the user base would find funny. RLHF essentially provides a mathematically crowdsourced alternative: prompt the LLM to generate jokes and have human testers rate their quality. These ratings can be used to train a *reward model* to predict the kinds of jokes that will receive positive feedback, and in turn that reward model can be use to train the LLM through reinforcement learning.

More practically, RLHF aims to address existential challenges of LLMs, like hallucinations, reflecting societal biases inherent in training data or dealing with rude or adversarial user inputs.

Common fine-tuning use cases

Fine-tuning can be used for a wide range of purposes, from customizing to supplementing the model's core knowledge to extending the model to entirely new tasks and domains.

- **Customizing style**: Models can be fine-tuned to reflect a brand's desired tone, from implementing complex behavioral patterns and idiosyncratic illustration styles to simple modifications like beginning each exchange with a polite salutation.
- **Specialization**: The general linguistic abilities of LLMs can be honed for specific tasks. For example, Meta's Llama 2 models were released as base foundation models, chatbot-tuned variants (Llama-2-chat) and code-tuned variants (Code Llama).
- **Adding domain-specific knowledge**: While LLMs are pre-trained on a massive corpus of data, they are not omniscient. Using additional training samples to supplement the base model's knowledge is particularly relevant in legal, financial or medical settings, which typically entail use of specialized, esoteric vocabulary that may not have been sufficiently represented in pre-training.
- **Few-shot learning**: Models that already have strong generalized knowledge can often be fine-tuned for more specific classification texts using comparatively few demonstrative examples.
- **Addressing edge cases**: You may want your model to handle certain situations that are unlikely to have been covered in pre-training in a specific way. Fine-tuning a model on labeled examples of such situations is an effective way to ensure they are dealt with appropriately.
- **Incorporating proprietary data**: Your company may have its own proprietary data pipeline, highly relevant to your specific use case. Fine-tuning allows this knowledge to be incorporated into the model without having to train it from scratch.

Author

Dave Bergmann
Senior Staff Writer, AI Models
IBM Think

Resources

Training
Level up your ML expertise
Learn fundamental concepts and build your skills with hands-on labs, courses, guided projects, trials and more.
Explore ML courses →

Ebook
Unlock the power of generative AI + ML
Learn how to confidently incorporate generative AI and machine learning into your business.
Read the ebook →

Techsplainers Podcast
Machine learning explained
Techsplainers by IBM breaks down the essentials of machine learning, from key concepts to real-world use cases. Clear, quick episodes help you learn the fundamentals fast.
Listen now →

Guide
Put AI to work: Driving ROI with gen AI
Want to get a better return on your AI investments? Learn how scaling gen AI in key areas drives change by helping your best minds build and deliver innovative new solutions.
Read the guide →

1 / 2

Related solutions

IBM® watsonx Orchestrate™
Easily design scalable AI assistants and agents, automate repetitive tasks and simplify complex processes with IBM® watsonx Orchestrate™.
Explore watsonx Orchestrate →

AI for developers
Move your applications from prototype to production with the help of our AI development solutions.
Explore AI development tools →

AI consulting and services
Reinvent critical workflows and operations by adding AI to maximize experiences, real-time decision-making and business value.
Explore AI services →

Take the next step
Whether you choose to customize pre-built apps and skills or build and deploy custom agentic services using an AI studio, the IBM watsonx platform has you covered.
Explore watsonx Orchestrate →
Explore watsonx.ai →

Footnotes

External links to ibm.com

[1] "Big Self-Supervised Models are Strong Semi-Supervised Learners" . arXiv. October 26, 2020

[2] "CSS-LM: A Contrastive Framework for Semi-supervised Fine-tuning of Pre-trained Language Models" . arXiv. March 2, 2021

[3] "On the Effectiveness of Parameter-Efficient Fine-Tuning" . arXiv. November 28, 2022

[4] "BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models" . arXiv. June 18, 2021 (last updated: September 5, 2022)

[5] "Scaling Sparse Fine-Tuning to Large Language Models" . arXiv. February 2, 2024

[6] "Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning" . arXiv. March 28, 2023

[7] "Parameter-Efficient Transfer Learning for NLP" . arXiv. June 13, 2019

IBM
Discover: Products, Consulting services, Industries, Case studies, Financing, Research
Connect: Business partners, Documentation, Events, Newsletters, Support, TechXchange community
About: Overview, Careers, Investor relations, Leadership, Newsroom, Security, privacy and trust
Follow: LinkedIn, X, Instagram, YouTube, Podcasts
Contact IBM Privacy Terms of use Accessibility Cookie Preferences